Status	Finished
Started	Friday, 30 October 2026, 3:35 PM
Completed	Friday, 30 October 2026, 3:40 PM
Duration	4 mins 45 secs
Grade	**15.00** out of 15.00 (**100**%)

Question **1**

Complete

Mark 1.00 out of 1.00

The process of placing or rearranging a collection of elements into a particular order is known as

- a. Sorting
- b. Searching
- c. Rearranging
- d. Merging

The correct answer is: Sorting

Question **2**

Complete

Mark 1.00 out of 1.00

What is a significant characteristic of Bubble Sort?

- a. It uses the heap data structure
- b. It recursively sorts subproblems
- c. It divides the list into sublists
- d. It bubbles up the largest element in each pass

The correct answer is: It bubbles up the largest element in each pass

Question **3**

Complete

Mark 1.00 out of 1.00

Two-way merge sort algorithm is used to sort the following elements in ascending order.
200,470,150,80,90,40,400,300,120,70
What is the order of these elements after second pass of the merge sort algorithm?

a. 40,70,80,90,120,150,200,300,400,470
b. 80,150,200,470,40,90,300,400,70,120
c. 40,80,90,150,200,300,400,470,70,120
d. 200,470,80,150,40,90,300,400,70,120

The correct answer is:
80,150,200,470,40,90,300,400,70,120

Question **4**

Complete

Mark 1.00 out of 1.00

What is a key disadvantage of Bubble Sort compared to more advanced algorithms like Merge Sort?

a. Bubble Sort does not guarantee sorted order
b. Bubble Sort is difficult to implement
c. Bubble Sort is less efficient for large lists
d. Bubble Sort cannot handle duplicate elements

The correct answer is: Bubble Sort is less efficient for large lists

Question **5**

Complete

Mark 1.00 out of 1.00

What is sorting in the context of computer science?

a. Searching for data in a list
b. Arranging data in a particular format
c. Inserting data into a list
d. Deleting data from a list

The correct answer is: Arranging data in a particular format

Question **6**

Complete

Mark 1.00 out of 1.00

What is the primary benefit of using sorting algorithms in programming?

- a. Provides a basis for other algorithms to work efficiently
- b. Makes data harder to manage
- c. Decreases the efficiency of algorithms
- d. Makes code execution slower

The correct answer is: Provides a basis for other algorithms to work efficiently

Question **7**

Complete

Mark 1.00 out of 1.00

Which of the following is not an in-place sorting algorithm?

- a. Merge sort
- b. Heap sort
- c. Selection sort
- d. Quick sort

The correct answer is:
Merge sort

Question **8**

Complete

Mark 1.00 out of 1.00

Which of the following is a key reason for the importance of sorting algorithms?

- a. Sorting is rarely used in programming
- b. Sorting decreases the efficiency of selection operations
- c. Sorting makes it harder to search for items
- d. Sorting helps in finding duplicates quickly

The correct answer is: Sorting helps in finding duplicates quickly

Question **9**

Complete

Mark 1.00 out of 1.00

Why is it advantageous to sort data before performing duplicate analysis?

- a. It allows for quicker identification of duplicates
- b. It makes the analysis slower
- c. It complicates the analysis process
- d. It has no effect on the analysis process

The correct answer is: It allows for quicker identification of duplicates

Question **10**

Complete

Mark 1.00 out of 1.00

What is one of the key advantages of using the built-in sorted() function in Python?

- a. It sorts data out of the box efficiently
- b. It is less efficient than custom sorting algorithms
- c. It requires external libraries
- d. It only works with integer arrays

The correct answer is: It sorts data out of the box efficiently

Question **11**

Complete

Mark 1.00 out of 1.00

_____is putting an element in the appropriate place in a sorted list yields a larger sorted order list.

- a. Selection
- b. Distribution
- c. Insertion
- d. Extraction

The correct answer is:
Insertion

Question **12**

Complete

Mark 1.00 out of 1.00

How does Merge Sort achieve its efficiency?

- a. By breaking the input into smaller parts and merging them
- b. By comparing elements sequentially
- c. By using the bubble-up method
- d. By sorting data in a single pass

The correct answer is: By breaking the input into smaller parts and merging them

Question **13**

Complete

Mark 1.00 out of 1.00

What is the primary advantage of the divide-and-conquer approach in sorting algorithms?

- a. It simplifies the sorting process by using only one pass
- b. It allows for efficient parallel processing and sorting of data
- c. It only works on small datasets
- d. It avoids the need for recursion

The correct answer is: It allows for efficient parallel processing and sorting of data

Question **14**

Complete

Mark 1.00 out of 1.00

Which of the following best describes the process of Merge Sort?

- a. It divides the list into two halves, sorts each half, and then merges them
- b. It compares adjacent elements and swaps them if necessary
- c. It builds a sorted array one element at a time
- d. It repeatedly finds the minimum element and moves it to the sorted part of the list

The correct answer is: It divides the list into two halves, sorts each half, and then merges them

Question **15**

Complete

Mark 1.00 out of 1.00

What is a characteristic of the merge sort algorithm?

a. It is less efficient than bubble sort
b. It sorts data using a single pass
c. It does not require recursion
d. It is based on the divide-and-conquer approach

The correct answer is: It is based on the divide-and-conquer approach

Status	Finished
Started	Friday, 30 October 2026, 4:00 PM
Completed	Friday, 30 October 2026, 4:08 PM
Duration	7 mins 55 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order. You read an list of numbers. You need to arrange the elements in ascending order and print the result. The sorting should be done using bubble sort.

Input Format: The first line reads the number of elements in the array. The second line reads the array elements one by one.

Output Format: The output should be a sorted list.

For example:

Input	Result
6 3 4 8 7 1 2	1 2 3 4 7 8
5 4 5 2 3 1	1 2 3 4 5

Answer: (penalty regime: 0 %)

```
def bubble_sort():
    # Read the number of elements
    n = int(input())

    # Read the space-separated array elements
    arr = list(map(int, input().split()))

    # Perform Bubble Sort algorithm
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                # Swap if adjacent elements are in wrong order
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    # Print the sorted elements separated by space
    print(*arr)

if __name__ == "__main__":
    bubble_sort()
```

	Input	Expected	Got	
✓	6 3 4 8 7 1 2	1 2 3 4 7 8	1 2 3 4 7 8	✓
✓	6 9 18 1 3 4 6	1 3 4 6 9 18	1 3 4 6 9 18	✓
✓	5 4 5 2 3 1	1 2 3 4 5	1 2 3 4 5	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

The problem is that we want to reverse a array in O(N) linear time complexity and we want the algorithm to be in-place as well!
For example: input is [1,2,3,4,5] then the output is [5,4,3,2,1]
Input
5
1 2 3 4 5
Output
5 4 3 2 1

For example:

Input	Result
5 1 2 3 4 5	5 4 3 2 1

Answer: (penalty regime: 0 %)

```
def reverse_array():
    # Read the number of elements
    n = int(input())

    # Read the array elements
    arr = list(map(int, input().split()))

    # Two-pointer approach
    left = 0
    right = n - 1

    # Swap elements moving towards the center
    while left < right:
        arr[left], arr[right] = arr[right], arr[left]
        left += 1
        right -= 1

    # Print the reversed array
    print(*arr)

if __name__ == "__main__":
    reverse_array()
```

	Input	Expected	Got	
✓	5 1 2 3 4 5	5 4 3 2 1	5 4 3 2 1	✓
✓	10 0 2 4 6 8 1 3 5 7 9	9 7 5 3 1 8 6 4 2 0	9 7 5 3 1 8 6 4 2 0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Given an integer array `nums`, return *an integer array* `counts` *where* `counts[i]` *is the number of smaller elements to the right of* `nums[i]`.

Example 1:

```
Input: nums = [5,2,6,1]
Output: [2,1,1,0]
Explanation:
To the right of 5 there are 2 smaller elements (2 and 1).
To the right of 2 there is only 1 smaller element (1).
To the right of 6 there is 1 smaller element (1).
To the right of 1 there is 0 smaller element.
```

Example 2:

```
Input: nums = [-1]
Output: [0]
```

Example 3:

```
Input: nums = [-1,-1]
Output: [0,0]
```

Constraints:

- `1 <= nums.length <= 10^5`
- `-10^4 <= nums[i] <= 10^4`

For example:

Test	Result
`print(countSmaller([5,2,6,1]))`	`[2, 1, 1, 0]`
`print(countSmaller([-1]))`	`[0]`

Answer: (penalty regime: 0 %)

Reset answer

```python
def countSmaller(nums):
    n = len(nums)
    counts = [0] * n
    # Store tuples of (number, original_index)
    arr = [(num, i) for i, num in enumerate(nums)]

    def merge_sort(enum):
        mid = len(enum) // 2
        if mid:
            left, right = merge_sort(enum[:mid]), merge_sort(enum[mid:])
            for i in range(len(enum) - 1, -1, -1):
                # If right array is empty or element in left is greater than element in right
                if not right or (left and left[-1][0] > right[-1][0]):
                    # Every remaining element in 'right' is smaller than left[-1][0]
                    counts[left[-1][1]] += len(right)
                    enum[i] = left.pop()
                else:
                    enum[i] = right.pop()
        return enum

    merge_sort(arr)
    return counts
```

	Test	Expected	Got	
✓	`print(countSmaller([5,2,6,1]))`	`[2, 1, 1, 0]`	`[2, 1, 1, 0]`	✓
✓	`print(countSmaller([50,20,60,10]))`	`[2, 1, 1, 0]`	`[2, 1, 1, 0]`	✓
✓	`print(countSmaller([-1]))`	`[0]`	`[0]`	✓
✓	`print(countSmaller([-1, -1]))`	`[0, 0]`	`[0, 0]`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Given an listof integers, sort the array in ascending order using the *Bubble Sort* algorithm above. Once sorted, print the following three lines:

1. List is sorted in numSwaps swaps., where numSwaps is the number of swaps that took place.
2. First Element: firstElement, the *first* element in the sorted list.
3. Last Element: lastElement, the *last* element in the sorted list.

For example, given a worst-case but small array to sort: a=[6,4,1]. It took 3 swaps to sort the array. Output would be

```
Array is sorted in 3 swaps.
```

```
First Element: 1
```

```
Last Element: 6
```

Input Format

The first line contains an integer,n , the size of the list a .
The second line contains n, space-separated integers a[i].

Constraints

· 2<=n<=600

· 1<=a[i]<=$2x10^6$.

Output Format

You must print the following three lines of output:

1. List is sorted in numSwaps swaps., where numSwaps is the number of swaps that took place.
2. First Element: firstElement, the *first* element in the sorted list.
3. Last Element: lastElement, the *last* element in the sorted list.

Sample Input 0

3

1 2 3

Sample Output 0

List is sorted in 0 swaps.

First Element: 1

Last Element: 3

For example:

Input	Result
3 3 2 1	List is sorted in 3 swaps. First Element: 1 Last Element: 3
5 1 9 2 8 4	List is sorted in 4 swaps. First Element: 1 Last Element: 9

Answer: (penalty regime: 0 %)

```
def count_swaps():
    n = int(input())
    a = list(map(int, input().split()))
```

```
    a = list(map(int, input().split()))

    num_swaps = 0

    for i in range(n):
        for j in range(0, n - 1):
            # Swap adjacent elements if they are in decreasing order
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                num_swaps += 1

    print(f"List is sorted in {num_swaps} swaps.")
    print(f"First Element: {a[0]}")
    print(f"Last Element: {a[-1]}")

if __name__ == "__main__":
    count_swaps()
```

	Input	Expected	Got	
✓	3 3 2 1	List is sorted in 3 swaps. First Element: 1 Last Element: 3	List is sorted in 3 swaps. First Element: 1 Last Element: 3	✓
✓	5 1 9 2 8 4	List is sorted in 4 swaps. First Element: 1 Last Element: 9	List is sorted in 4 swaps. First Element: 1 Last Element: 9	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Given an array of integers `nums`, sort the array in **increasing** order based on the frequency of the values. If multiple values have the same frequency, sort them in **decreasing** order.

Print the *sorted array*.

Example 1:

```
Input:
6
1 1 2 2 2 3

Output:
3 1 1 2 2 2

Explanation: '3' has a frequency of 1, '1' has a frequency of 2, and '2' has a frequency of 3.
```

Example 2:

```
Input:
5
2 3 1 3 2
Output:
1 3 3 2 2
Explanation: '2' and '3' both have a frequency of 2, so they are sorted in decreasing order.
```

Example 3:

```
Input:
9
-1 1 -6 4 5 -6 1 4 1
Output:
5 -1 4 4 -6 -6 1 1 1
```

Constraints:

- `1 <= nums.length <= 100`
- `-100 <= nums[i] <= 100`

For example:

Input	Result
6 1 1 2 2 2 3	3 1 1 2 2 2
5 2 3 1 3 2	1 3 3 2 2

Answer: (penalty regime: 0 %)

```
from collections import Counter

def frequency_sort():
    n = int(input())
    nums = list(map(int, input().split()))

    # Count frequencies of each number
    counts = Counter(nums)

    # Sort key: (frequency in ascending order, value in descending order)
    nums.sort(key=lambda x: (counts[x], -x))
```

```
    # Print the sorted array separated by space
    print(*nums)

if __name__ == "__main__":
    frequency_sort()
```

	Input	Expected	Got	
✓	6 1 1 2 2 2 3	3 1 1 2 2 2	3 1 1 2 2 2	✓
✓	5 2 3 1 3 2	1 3 3 2 2	1 3 3 2 2	✓
✓	9 -1 1 -6 4 5 -6 1 4 1	5 -1 4 4 -6 -6 1 1 1	5 -1 4 4 -6 -6 1 1 1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

To find the frequency of numbers in a list and display in sorted order.

Constraints:

1<=n, arr[i]<=100

Input:

1 68 79 4 90 68 1 4 5

output:

1 2

4 2

5 1

68 2

79 1

90 1

For example:

Input	Result
4 3 5 3 4 5	3 2 4 2 5 2

Answer: (penalty regime: 0 %)

```
from collections import Counter

def frequency_count():
    # Read the space-separated numbers
    arr = list(map(int, input().split()))

    # Count frequency of each number
    counts = Counter(arr)

    # Sort by the numbers (keys) in ascending order and print
    for num in sorted(counts.keys()):
        print(f"{num} {counts[num]}")

if __name__ == "__main__":
    frequency_count()
```

	Input	Expected	Got	
✓	4 3 5 3 4 5	3 2 4 2 5 2	3 2 4 2 5 2	✓

	Input	Expected	Got	
✓	12 4 4 4 2 3 5	2 1 3 1 4 3 5 1 12 1	2 1 3 1 4 3 5 1 12 1	✓
✓	5 4 5 4 6 5 7 3	3 1 4 2 5 3 6 1 7 1	3 1 4 2 5 3 6 1 7 1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Write a Python program to sort a list of elements using the merge sort algorithm.

For example:

Input	Result
5 6 5 4 3 8	3 4 5 6 8

Answer: (penalty regime: 0 %)

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left_half = arr[:mid]
        right_half = arr[mid:]

        # Recursive call on each half
        merge_sort(left_half)
        merge_sort(right_half)

        i = j = k = 0

        # Merge the two sorted halves
        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1

        # Copy any remaining elements of left_half
        while i < len(left_half):
            arr[k] = left_half[i]
            i += 1
            k += 1

        # Copy any remaining elements of right_half
        while j < len(right_half):
            arr[k] = right_half[j]
            j += 1
            k += 1

def main():
    n = int(input())
    arr = list(map(int, input().split()))

    merge_sort(arr)

    print(*arr)

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	5 6 5 4 3 8	3 4 5 6 8	3 4 5 6 8	✓
✓	9 14 46 43 27 57 41 45 21 70	14 21 27 41 43 45 46 57 70	14 21 27 41 43 45 46 57 70	✓
✓	4 86 43 23 49	23 43 49 86	23 43 49 86	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Given an integer array `nums` sorted in **non-decreasing** order, return *an array of **the squares of each number** sorted in non-decreasing order*.

Example 1:

```
Input: nums = [-4,-1,0,3,10]
Output: [0,1,9,16,100]
Explanation: After squaring, the array becomes [16,1,0,9,100].
After sorting, it becomes [0,1,9,16,100].
```

Example 2:

```
Input: nums = [-7,-3,2,3,11]
Output: [4,9,9,49,121]
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^4 <= nums[i] <= 10^4`
- `nums` is sorted in **non-decreasing** order.

For example:

Test	Result
`print(sortedSquares([-4,-1,0,3,10]))`	`[0, 1, 9, 16, 100]`

Answer: (penalty regime: 0 %)

Reset answer

```
def sortedSquares(nums):
    n = len(nums)
    result = [0] * n
    left = 0
    right = n - 1

    # Fill result array from right to left (largest to smallest square)
    for i in range(n - 1, -1, -1):
        if abs(nums[left]) > abs(nums[right]):
            result[i] = nums[left] ** 2
            left += 1
        else:
            result[i] = nums[right] ** 2
            right -= 1

    return result

```

	Test	Expected	Got	
✓	`print(sortedSquares([-4,-1,0,3,10]))`	`[0, 1, 9, 16, 100]`	`[0, 1, 9, 16, 100]`	✓

	Test	Expected	Got	
✓	`print(sortedSquares([-7,-3,2,3,11]))`	`[4, 9, 9, 49, 121]`	`[4, 9, 9, 49, 121]`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Given an array of integers `arr`, replace each element with its rank.

The rank represents how large the element is. The rank has the following rules:

- Rank is an integer starting from 1.
- The larger the element, the larger the rank. If two elements are equal, their rank must be the same.
- Rank should be as small as possible.

Example 1:

```
Input: arr = [40,10,20,30]
Output: [4,1,2,3]
Explanation: 40 is the largest element. 10 is the smallest. 20 is the second smallest. 30 is the third smallest.
```

Example 2:

```
Input: arr = [100,100,100]
Output: [1,1,1]
Explanation: Same elements share the same rank.
```

Example 3:

```
Input: arr = [37,12,28,9,100,56,80,5,12]
Output: [5,3,4,2,8,6,7,1,3]
```

Constraints:

- `0 <= arr.length <= 10^5`
- `-10^9 <= arr[i] <= 10^9`

For example:

Test	Result
`print(arrayRankTransform([40,10,20,30]))`	`[4, 1, 2, 3]`

Answer: (penalty regime: 0 %)

Reset answer

```
def arrayRankTransform(arr):
    # Sort unique elements and map each to its 1-based rank
    rank = {val: i + 1 for i, val in enumerate(sorted(set(arr)))}

    # Replace each element in arr with its rank
    return [rank[num] for num in arr]


```

	Test	Expected	Got	
✓	`print(arrayRankTransform([40,10,20,30]))`	`[4, 1, 2, 3]`	`[4, 1, 2, 3]`	✓
✓	`print(arrayRankTransform([100,100,100]))`	`[1, 1, 1]`	`[1, 1, 1]`	✓
✓	`print(arrayRankTransform([37,12,28,9,100,56,80,5,12]))`	`[5, 3, 4, 2, 8, 6, 7, 1, 3]`	`[5, 3, 4, 2, 8, 6, 7, 1, 3]`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Objective:

Develop a Python program to find the k-th maximum value in a given list of integers. The program should handle various edge cases, including lists with duplicate values, empty lists, and invalid values of k. The k-th maximum value refers to the k-th largest distinct element in the list.

Background:

Finding the k-th maximum value in a list is a common problem in computer science, often encountered in fields like data analysis, competitive programming, and software development. This problem requires an understanding of sorting algorithms, data structures, and efficient problem-solving techniques. By solving this problem, one gains insights into how to handle large datasets and optimize performance in practical applications.

Problem Description:

Given a list of integers, the task is to determine the k-th maximum value in the list. The program should meet the following requirements:

1. **Input:**
    - A list of integers, which can contain both positive and negative values.
    - An integer k, representing the position of the maximum value to find.
2. **Output:**
    - The k-th maximum value in the list.
    - If k is greater than the number of distinct elements in the list or if the list is empty, the program should return an appropriate message indicating the error.

Constraints:

- The list may contain duplicate values.
- The value of k should be a positive integer.
- The list may contain up to 10^6 elements, and each element can be as large as 10^9 in magnitude.

Examples:

Consider the following examples for better understanding:

1. **Example 1:**
    - **Input:** list = [3, 1, 5, 4, 2], k = 2
    - **Output:** 4
    - **Explanation:** The distinct elements in the list are [1, 2, 3, 4, 5]. The 2nd maximum value is 4.
2. **Example 2:**
    - **Input:** list = [7, 7, 7, 7, 7], k = 1
    - **Output:** 7
    - **Explanation:** The distinct elements in the list are [7]. The 1st maximum value is 7.
3. **Example 3:**
    - **Input:** list = [2, 1, 2, 1, 2], k = 3
    - **Output:** -1
    - **Explanation:** The distinct elements in the list are [1, 2]. There is no 3rd maximum value.

For example:

Input	Result
5 3 1 5 4 2 2	4

Input	Result
6 7 7 7 7 7 7 1	7
10 2 1 2 1 2 1 2 1 2 1 3	-1

Answer: (penalty regime: 0 %)

```
def find_kth_max():
    try:
        # Read the number of elements in the list
        n = int(input())

        # Read the list elements
        lst = list(map(int, input().split()))

        # Read the value of k
        k = int(input())

        # Extract distinct elements and sort them in descending order
        distinct_sorted = sorted(set(lst), reverse=True)

        # Check if k is valid and within range
        if 1 <= k <= len(distinct_sorted):
            print(distinct_sorted[k - 1])
        else:
            print(-1)

    except (ValueError, EOFError):
        print(-1)

if __name__ == "__main__":
    find_kth_max()
```

	Input	Expected	Got	
✓	5 3 1 5 4 2 2	4	4	✓
✓	6 7 7 7 7 7 7 1	7	7	✓
✓	10 2 1 2 1 2 1 2 1 2 1 3	-1	-1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.